



A tactical RPG where disgraced gods fight to restore the sun across a collapsing Egyptian underworld.

- **Developer:** Karthik
- **Role:** Solo Developer, Game Designer, Technical Artist
- **Project Type:** Passion Project / Engineering Case Study
- **Development Status:** Version 1.5 (Complete)
- **Tech Stack:** Python 3, Kivy Framework, Android OS Threading, Buildozer

01 — Executive Summary

The Missing Sun is a heavyweight mobile game engineered entirely from scratch using Python. By eschewing commercial, out-of-the-box game engines like Unity or Godot, this project serves as a comprehensive case study in custom application architecture, dynamic memory management, and full-stack mobile development using the Kivy framework.

Rather than relying on a repetitive, static gameplay loop, the engine operates as a highly dynamic state machine. The application intelligently routes the player through an 83-act branching narrative, seamlessly loading distinct sub-engines for interactive logic puzzles and tactical combat strictly when the story dictates.

The combat engine itself is a flexible, highly scalable system. It moves away from traditional fixed-party constraints, supporting variable skirmishes that range from intimate 1v1 duels up to massive, asymmetrical 8v12 battlefields.

Category	Details
Genre	Tactical RPG / Puzzle-Adventure / Visual Novel
Platform	Android Mobile (Compiled via Buildozer/NDK)
Game Flow	Dynamic Story-Driven State Routing (Narrative → Contextual Puzzles → Combat)
Combat Scale	Variable Turn-Based Encounters (Scales from 1v1 up to 8v12)
Playable Roster	14 Unique Playable Entities (6 featuring alternate state forms)
Enemy Roster	Expansive, separate bestiary of CPU-exclusive gods and mutated creatures
Engine	Custom Architecture (Python 3 / Kivy)

02 — Game Concept

World Premise & Stakes

The narrative is rooted in a dark, high-stakes interpretation of Egyptian mythology. The Mandjet (the Solar Barque) has been ambushed by an unknown, catastrophic force, plunging the cosmos into an eternal eclipse.

The player commands the disgraced protectors known as the "Eye of Ra." They are tasked with navigating the deadly, shifting wastelands of the Duat to locate and reforge the three fractured pieces of the Sun King's soul (Khepri, Atum, and Khnum). The pressure is absolute: they must restore the true Sun not just before the Great Devourer, Apep, reaches full strength, but before a corrupted, panicked divine council permanently appoints a false Sun God to the vacant throne.

Art Direction & Creative Pipeline

To achieve a AAA-style visual and auditory presentation on a solo developer budget, the project merged native Python programming with a highly specific, AI-accelerated creative pipeline. I acted as both Technical and Art Director, ensuring all assets adhered to a cohesive dark-mythology aesthetic.

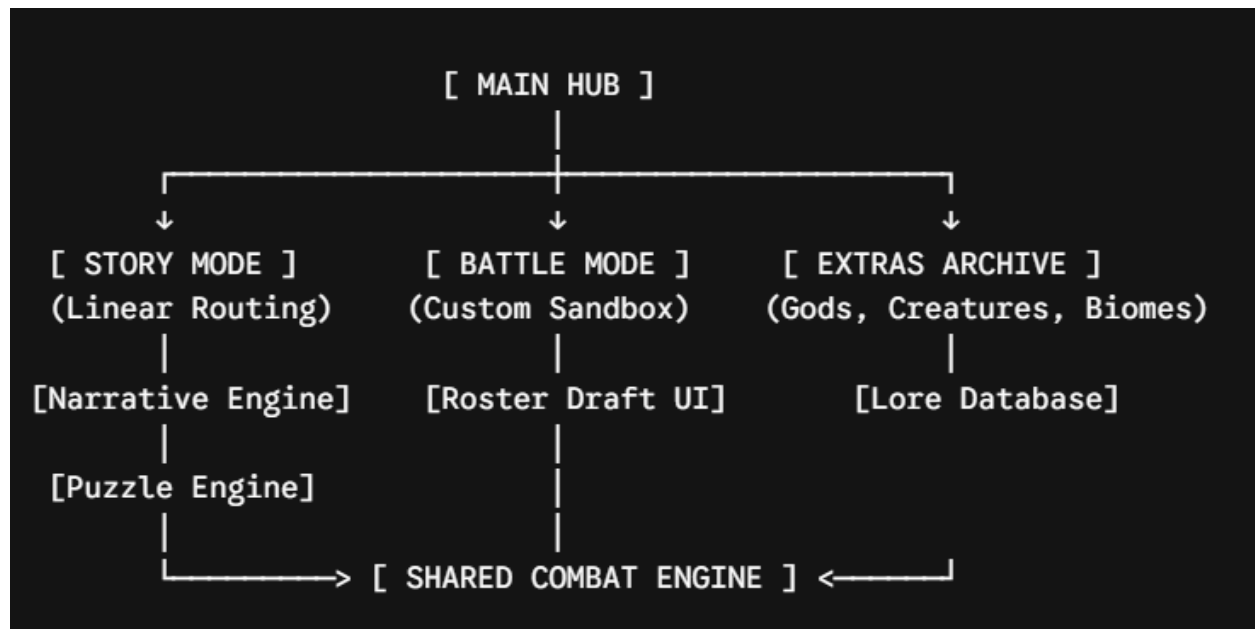
- **Ibis Paint (Character Design & Line Art):** All core character designs, foundational visual assets, and custom UI elements were originally conceptualized and hand-drawn to establish the game's unique visual identity.
- **ChatGPT (Animation & Environment Generation):** Rather than generating raw concepts, AI was utilized as an animation and background assistant. It was directed to process my original character designs and generate fluid, varied pose animations for the sprites, as well as render the highly detailed, atmospheric background environments for the visual novel scenes.
- **Suno AI (Audio Engineering):** The game's auditory experience was curated and generated using AI audio tools, providing each unique biome and tactical combat encounter with its own professional-grade, thematic soundscape.

Here is the expanded and highly detailed breakdown for Sections 04, 05, and 06, implementing your exact structural corrections and expanding on the technical design of the engine architecture.

03 — Core Gameplay & Engine Architecture

On-Demand Engine Architecture

The Missing Sun rejects the traditional monolithic game loop in favor of a highly modular, event-driven state machine managed by Kivy's `ScreenManager`. Rather than keeping all game logic running simultaneously, the application instances and destroys distinct, isolated sub-engines strictly on demand.



The Core Sub-Engines:

- **The Narrative Engine:** A deeply integrated visual novel system that parses the 83-act script. It handles asynchronous background loading of sprite poses and environmental art generated during the creative pipeline, managing text flow and audio transitions.
- **The Puzzle Engine:** Called contextually during specific story acts. This engine temporarily suspends the narrative state to load custom, algorithmically driven logic modules (e.g., node-based flow algorithms, rotational trigonometry puzzles).
- **The Shared Combat Engine:** A centralized, highly complex tactical module. This exact same engine is utilized by both Story Mode (where enemy parameters and team sizes are dictated by the plot) and Battle Mode (where parameters are fed via the player's custom Roster Draft UI). This "write once, use everywhere" architecture ensures that any updates or balancing tweaks to combat inherently apply across the entire application without redundant code.
- **The Extras Archive:** A supplementary, dynamically populated database allowing players to explore the lore of the biomes, gods, and creatures outside of the main loop.

04 — Characters & Combat

Asymmetrical Roster Scale:

The game features a massive, highly varied entity database designed to support David-vs-Goliath style encounters.

- **Player Roster:** A focused, tightly balanced pool of **14 playable characters**. Out of this core group, 6 characters possess secondary and tertiary alternate forms, drastically altering their tactical utility.
- **CPU Roster:** A sprawling bestiary of approximately **48 unique enemies** (excluding alternate forms), ranging from corrupted gods to mutated biome-specific abominations.

Tactical Combat Mechanics:

- **Variable Encounter Sizing:** The combat engine is built to be dynamically scalable. It safely handles memory and UI rendering for anything from an intimate 1v1 duel up to a massive 8v12 asymmetrical war.
- **Dynamic Speed Queues:** The engine discards the rigid "Player Phase / Enemy Phase" binary. It recalculates an `active_queue` array at the start of every round, sorting up to 20 entities purely by their internal Speed (`sp`) stats to determine absolute attack priority.
- **State & Array Management:** Built entirely on Object-Oriented Programming (OOP) principles, the engine tracks multidimensional arrays to execute Area-of-Effect (AoE) targeting logic, calculate multi-round ability cooldowns, process Damage-over-Time (DoTs), and evaluate persistent stat buffs across prolonged encounters.

05 — Progression

Narrative Escalation & Form Evolution

To maintain a tight, cinematic pacing, *The Missing Sun* deliberately avoids artificial grinding, arbitrary currency treadmills, or microtransaction-style unlocking systems.

Progression is strictly tied to narrative milestones and tactical evolution:

- **Story Progression:** The primary driver of the game. Advancing through the 83 acts pushes the player deeper through the Duat's distinct biomes (The Mirage Temple, The Cursed Desert, The Deep Duat), escalating the difficulty and complexity of the embedded puzzle engines.
- **Form Acquisition:** True mechanical progression is achieved through narrative milestones that awaken alternate character states. Unlocking a "Solaris" or "Chaos" form shifts a character's base stats, movesets, and visual sprites, requiring the player to constantly adapt their combat strategies as the CPU roster grows increasingly hostile.

06 — Technical Architecture

The State Controller

The application architecture is centered around Kivy's `ScreenManager`, functioning as an advanced, event-driven state controller. Rather than pre-loading the entire game into memory, the architecture utilizes a custom "Factory" method (`build_screen_by_name`). This method dynamically instantiates specific classes (e.g., `KhnumRitual`, `BattleScreen`, `NarrativeScreen`) strictly when the routing sequence calls for them.

Decoupled Logic & Shared Engines

To maintain a DRY (Don't Repeat Yourself) codebase, major systems are heavily decoupled:

- The **Combat Engine** is completely agnostic to how it is launched. Whether it receives its arrays (Player/CPU rosters) from the linear Story Mode or the custom Battle Draft UI, it processes the data identically.
- The **Puzzle Engine** acts as an interrupt. When a puzzle node is reached in the `GAME_SEQUENCE` array, the narrative state is temporarily suspended, control is handed over to the specific puzzle class, and upon solving, control is returned to the narrative router.

Animation Synchronization

To prevent the CPU from bottlenecking during screen swaps, the routing logic utilizes a strict timing delay. When a state change is triggered, the engine allows Kivy's `FadeTransition` exactly 1.2 seconds to complete its visual animation smoothly *before* the CPU is allowed to begin assembling the widgets and layouts for the upcoming screen.

07 — Development Process

The project was built using an iterative, agile-style release cycle, focusing first on core stability before expanding features and optimizing performance.

- **Version 1.0 (The Core Game):** Development of the foundational mechanics. This included the Narrative visual novel engine, the complete 83-act story sequence, the integration of the interactive puzzles, and the automated Combat Engine tailored strictly for the story encounters.
- **Version 1.1 (Lore Expansion):** Introduction of the *Extras Archive*. This update built the dynamic File I/O system, allowing the application to read `.txt` and `.png` data from the device at runtime to populate the Gods, Creatures, and Biomes databases.
- **Version 1.2 (Tactical Expansion):** Introduction of the dedicated *Battle Mode*. This update decoupled the Combat Engine from the story, implementing the UI grids and overlay logic for the custom 8v12 Roster Draft system.
- **Version 1.3 to 1.5 (Optimization & Polish):** A massive overhaul of the application's underlying performance. These versions focused entirely on total UI optimization, implementing asynchronous threading for asset loading, rewriting the memory management algorithms to

prevent crashes, and refining the cross-platform compilation toolchain to ensure the APK was production-ready.

08 — Major Challenges

Problem 1: Mobile RAM Limitations (OOM Crashes)

- **The Issue:** Rendering 83 story acts and 30 battle arenas laden with uncompressed `.png` and `.mp3` assets caused the application to quickly exceed Android's strict background RAM limits, resulting in fatal Out-Of-Memory (OOM) crashes. Kivy natively caches loaded screens indefinitely.
- **The Solution:** I engineered a proprietary "**Sliding Window**" **Garbage Collector**. The engine uses a predefined `GAME_SEQUENCE` array to track the player's position. It looks 2 screens ahead to prepare incoming assets, while aggressively targeting the immediate previous screen (`current_index - 1`). It does not simply remove the widget; it actively hooks into Kivy's internal `Cache` manager to forcefully purge the raw graphical textures (`kv.image`, `kv.texture`) and calls Python's `del` to command the garbage collector, keeping the RAM footprint exceptionally tiny.

Problem 2: Main Thread Freezes (ANR Warnings)

- **The Issue:** Kivy operates on a single main thread. Attempting to decode massive audio and image files during cross-fade screen transitions forced the UI to lock up, triggering Android's "App Not Responding" (ANR) warnings and destroying the user experience.
- **The Solution:** Graphics cannot be manipulated outside the main thread without crashing the OpenGL context. To bypass this, I implemented **Asynchronous OS-Level Caching**. Using Python's `threading` module, a background thread was dispatched to read the raw bytes of the upcoming asset files into the Android OS RAM *behind the scenes*. When Kivy's main UI thread eventually needs to build the screen, it pulls the assets instantaneously from ultra-fast RAM rather than slow flash storage, eliminating all transition latency.

Problem 3: The Toolchain Compilation Paradox

- **The Issue:** Compiling native Python to Android C-code resulted in severe compiler conflicts within the Buildozer toolchain. Automated pipelines pulling the newest `pip` update broke the internal `python-for-android` build scripts (`BuildDependencyInstallError`). However, attempting to switch to a patched development branch introduced a new graphics recipe (`libthorvg`) that crashed (`IndexError: list index out of range`) when paired with older Android NDK folder structures.
- **The Solution:** Executed strict, manual environment management to resolve the paradox. I modified the `buildozer.spec` configuration to utilize the patched `p4a.branch = develop` (bypassing the `pip` crash) while forcing the compiler to use the specific, newer Android NDK (`r28c`) that

possessed the exact C-compiler folder structures the new graphics recipe required. This successfully unjammed the pipeline and yielded a stable APK.

09 — Testing & Balancing

The "Iteration Tax" (Cross-Platform Compilation)

The most grueling and difficult aspect of this project was the testing cycle. Because Kivy behaves differently on a desktop simulator than it does on physical Android hardware (touch event handling, RAM constraints, screen scaling), the game could not be reliably tested on a PC.

Every single bug fix, UI adjustment, and balance tweak required a punishing iteration loop:

1. Writing and modifying the Python code (`main.py`).
2. Executing the Buildozer C-compiler pipeline (`buildozer android debug`) to package the dependencies and generate an Android package.
3. Sideload the compiled `.apk` onto a physical Android device.
4. Running on-device QA testing.
5. Returning to the Python codebase to adjust and repeat.

Asymmetrical Combat Balancing

This slow compilation loop made balancing the 8v12 Combat Engine highly challenging. With 14 distinct player characters (many possessing alternate stat-altering forms) facing off against a bestiary of 48 distinct CPU enemies, raw math was not enough; it required on-device playtesting.

- **The Problem:** Initial `.apk` builds revealed that if the CPU deployed all 12 units simultaneously, the sheer action-economy would mathematically overwhelm the player's 8-unit roster, regardless of strategy.
- **The Solution:** Rather than nerfing the CPU enemies, I utilized the testing data to implement an **Automated Reinforcement System**. The CPU now fields a frontline, with reserves held in an array. As front-line units faint, the engine automatically splices fresh reserves into the active combat layout, maintaining the high stakes without mathematically breaking the turn queue.

10 — Performance

Achieving a native, zero-latency feel on mobile devices using Python required aggressively overriding Kivy's default handling of assets and memory.

1. Eliminating UI Thread Blocking (Asynchronous Caching)

- **The Optimization:** Mobile processors bottleneck when attempting to read large files from flash storage while simultaneously drawing UI pixels. I utilized Python's `threading` module to intercept the asset pipeline. Background threads were dispatched to silently read the raw bytes of massive `.png` and `.mp3` files directly into Android's ultra-fast RAM *before* the player actually reached that screen.
- **The Impact:** By the time Kivy's main thread triggers the visual transition, the assets are already waiting in RAM. This completely eliminated "App Not Responding" (ANR) lockups and ensured 60 FPS screen transitions.

2. Strict Garbage Collection (The Sliding Window)

- **The Optimization:** Left to its own devices, Kivy will cache every loaded image indefinitely. I implemented a strict "Sliding Window" index manager that actively hunts down the previous screen state (`cleanup_index = current_index - 1`).
- **The Impact:** Instead of waiting for Python's passive garbage collector, the engine manually executes `Cache.remove()` on Kivy's core graphics matrices and forces a `del` command on the screen widget. This capped the application's RAM footprint, preventing Out-Of-Memory (OOM) crashes even during hours of continuous gameplay on older devices.

11 — Lessons Learned

Engineering: The Reality of Cross-Platform Toolchains

I learned that writing the game code is only 50% of mobile development. Compiling Python for Android requires a deep, uncompromising understanding of Linux environments, package managers, and C-compilers. Navigating Buildozer, overriding broken `pip` dependencies, and hard-pinning specific Android NDK architectures taught me how to manage and debug volatile CI/CD pipelines.

Production: The Importance of Decoupled Architecture

Because the `.py` to `.apk` compilation loop was so time-consuming, I learned the critical importance of modular architecture. By completely decoupling the Narrative, Puzzle, and

Combat engines, I ensured that tweaking a combat mechanic wouldn't accidentally break a story transition. This isolated state-machine approach saved the project from collapsing under its own weight during the later stages of development.

Game Design: Hardware-Appropriate Mechanics

I learned that forcing reflex-heavy action mechanics into a framework not designed for them results in a poor user experience. Instead, I pivoted the design to rely on "cognitive friction." By building puzzles based on graph-traversal logic, rotational trigonometry, and active meter-balancing, I was able to create deep, engaging gameplay that felt perfectly natural for touch-screens.

12 — Future Roadmap

While Version 1.0 successfully establishes the core engine and narrative, future iterations (Version 1.x) will focus on structural pacing and Quality of Life (QoL) enhancements tailored specifically for mobile user habits.

- **Campaign Segmentation:** Transitioning the current continuous 83-act linear campaign into a structured Chapter/Level system. This architectural shift will better accommodate short-burst mobile play sessions, allow for localized save-states, and enable players to replay specific battles without restarting the narrative.
- **Quality of Life (QoL) State Interrupts:** Engineering robust UI overlays to implement "Pause" functionality within the active Combat and Puzzle state machines, as well as a "Skip" feature within the Narrative Engine for players traversing previously cleared story nodes.
- **Persistent Save Architecture:** Expanding the current local environment to utilize SQLite or JSON data structures, tracking campaign progress, unlocked lore, and roster availability across multiple play sessions.

13 — My Contribution

Role: Solo Developer, Game Designer, and Technical Artist

As the sole architect and creator of *The Missing Sun*, I owned the end-to-end product lifecycle—from initial conceptualization and creative direction to systems engineering and cross-platform compilation.

- **Software Engineering & Architecture:** Single-handedly programmed the entire Python codebase using the Kivy framework. Designed the on-demand state-machine routing, engineered the "Sliding Window" garbage collector for RAM optimization, and implemented asynchronous OS-level threading to bypass single-thread UI bottlenecks.
- **Game & Systems Design:** Authored the 83-act narrative script and designed the underlying mathematics for the tactical 8v12 Combat Engine. Programmed over 10 custom logic and physics puzzles utilizing graph-traversal algorithms (BFS) and custom collision geometry.
- **DevOps & Deployment:** Managed the Linux-based CI/CD compilation toolchain. Handled package dependency resolution, Android NDK/SDK environment variables, and Buildozer `.spec` configurations to successfully package the Python code into a production-ready `.apk`.
- **Art & Audio Direction:** Hand-drew core visual assets and UI elements using Ibis Paint. Executed advanced prompt engineering via ChatGPT to generate dynamic sprite poses and atmospheric backgrounds, and utilized Suno AI to produce and curate the game's professional-grade original soundtrack.

14 — Links / Contact

- **Gameplay Video:** <https://github.com/vvk625/The-Missing-Sun.git>
- **GitHub Repository:** <https://github.com/vvk625/The-Missing-Sun.git>
- **Contact:** letsplayproductions0507@gmail.com